STAT248 Lecture 6 Simple Linear Regression (Coding)

1 Simple Linear Regression (Coding)

1.1 Linear Trend Plus Noise

数据生成:

考虑 linear trend plus noise:

$$y_t = 2 + 0.5t + \varepsilon_t$$

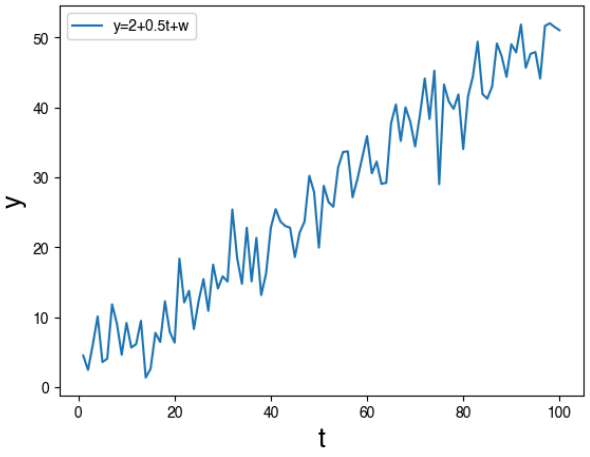
其中 $\varepsilon_t \overset{i.i.d.}{\sim} \mathcal{N}(0, 16)$

Python

```python
import numpy as np
from matplotlib import pyplot as plt
import statsmodels.api as sm
from statsmodels.graphics.tsaplots import plot_acf
np.random.seed(42)

t = np.arange(1, 101)
sigma = 4 # If sigma=1, standard normal
y = 2 + 0.5*t + sigma*np.random.randn(len(t))

plt.plot(t,y,label='y=2+0.5t+w')
plt.xlabel('t',fontsize=18)
plt.ylabel('y',fontsize=18)
plt.legend()
```



模型拟合:

使用 `statsmodels` 包来 fit simple model:

$$y_t = \beta_0 + \beta_1 t + \varepsilon_t$$

Python

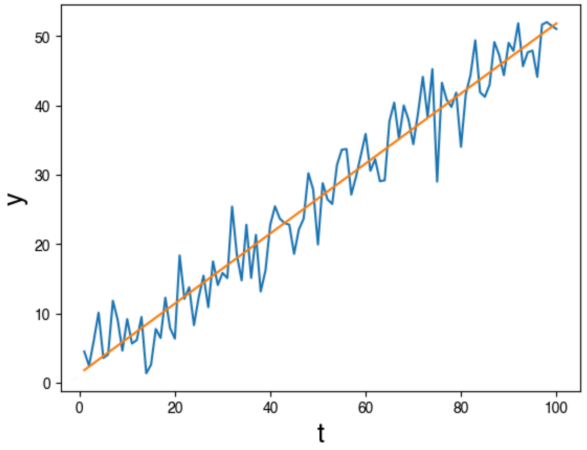
```python
T = sm.add_constant(t)
linreg_simple = sm.OLS(y, T).fit()
print(linreg_simple.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.942
Model:                            OLS   Adj. R-squared:                  0.942
Method:                 Least Squares   F-statistic:                     1601.
Date:                Tue, 03 Feb 2026   Prob (F-statistic):           1.62e-62
Time:                        19:51:25   Log-Likelihood:                -270.29
No. Observations:                 100   AIC:                             544.6
Df Residuals:                      98   BIC:                             549.8
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.3032      0.735      1.773      0.079      -0.155       2.762
x1             0.5056      0.013     40.010      0.000       0.480       0.531
==============================================================================
Omnibus:                        0.521   Durbin-Watson:                   2.042
Prob(Omnibus):                  0.771   Jarque-Bera (JB):                0.518
Skew:                          -0.167   Prob(JB):                        0.772
Kurtosis:                       2.885   Cond. No.                         117.
==============================================================================
```

可视化结果:

Python

```python
plt.plot(t,y)
plt.plot(t,linreg_simple.fittedvalues)
plt.xlabel('t',fontsize=18)
plt.ylabel('y',fontsize=18)
```



置信区间:

Python

```python
ci_linreg_simple = linreg_simple.conf_int(alpha=0.05)
print(ci_linreg_simple)
print(f'95% confidence interval for B0$: [{ci_linreg_simple[0,0]:.3f},
{ci_linreg_simple[0,1]:.3f}]')
print(f'95% confidence interval for B1$: [{ci_linreg_simple[1,0]:.3f},
{ci_linreg_simple[1,1]:.3f}]')
```

```
[[-0.15543644  2.76178753]
 [ 0.48049713  0.53064895]]
95% confidence interval for B0$: [-0.155, 2.762]
95% confidence interval for B1$: [0.480, 0.531]
```

模型诊断:

作出 residuals 及其 autocorrelation function 进行模型诊断:

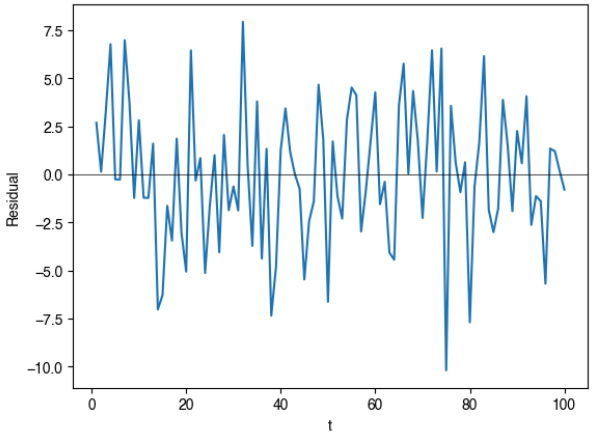
Python

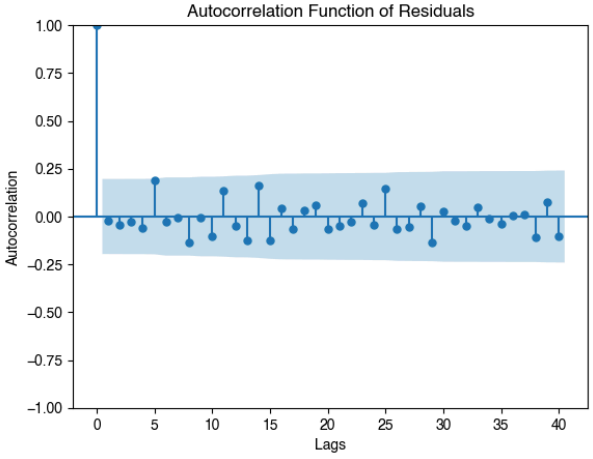
```python
residuals = linreg_simple.resid
```

```python

plt.figure()
plt.plot(t,residuals)
plt.axhline(0, color='k', linewidth=0.5) # Horizontal line at 0
plt.xlabel('t')
plt.ylabel('Residual')

plt.figure(figsize=(10, 5))
plot_acf(residuals, lags=40, title='Autocorrelation Function of Residuals')
plt.xlabel('Lags')
plt.ylabel('Autocorrelation')
plt.show()
```





1.2 Chicken Price Regression

模型设置:

使用 simple linear regression 来拟合 chicken price data:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

其中,

- y_i 为 time index i 时刻的 chicken price
- $x_i = i$

数据清洗

首先进行数据清洗和可视化:

Python

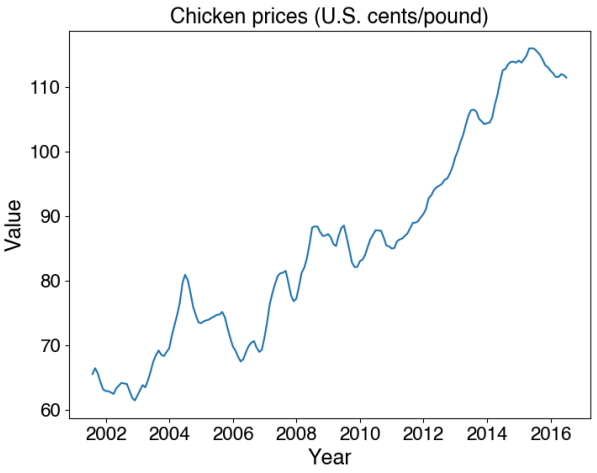
```python
import numpy as np
```

```python
from matplotlib import pyplot as plt
import pandas as pd
import statsmodels.api as sm
from statsmodels.graphics.tsaplots import plot_acf

#!pip install astsa
import astsa

chicken_data = astsa.load_chicken()
chicken_data['Time'] = pd.to_datetime(chicken_data['Time'])
chicken_data.set_index('Time', inplace=True)

plt.figure(figsize=(8,6))
plt.plot(chicken_data.index, chicken_data['Value'], label='Value')
plt.xlabel('Year', fontsize=18)
plt.ylabel('Value', fontsize=18)
plt.title('Chicken prices (U.S. cents/pound)', fontsize=18)
plt.xticks(fontsize=16)
plt.yticks(fontsize=16);
```



模型拟合:

Python

```python
yvec = np.array(chicken_data['Value'])
n = len(yvec) # number of time points
xvec = np.arange(1, n+1)
X = sm.add_constant(xvec)
linreg = sm.OLS(yvec, X).fit()
print(linreg.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.917
Model:                            OLS   Adj. R-squared:                  0.917
Method:                 Least Squares   F-statistic:                     1974.
Date:                Tue, 03 Feb 2026   Prob (F-statistic):          2.83e-98
Time:                        19:51:28   Log-Likelihood:                -532.83
No. Observations:                 180   AIC:                             1070.
Df Residuals:                     178   BIC:                             1076.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         58.5832      0.703     83.331      0.000      57.196      59.971
x1             0.2993      0.007     44.434      0.000       0.286       0.313
==============================================================================
Omnibus:                        9.576   Durbin-Watson:                   0.046
Prob(Omnibus):                  0.008   Jarque-Bera (JB):                4.204
Skew:                           0.018   Prob(JB):                        0.122
Kurtosis:                       2.252   Cond. No.                         210.
==============================================================================
```

置信区间:

Python

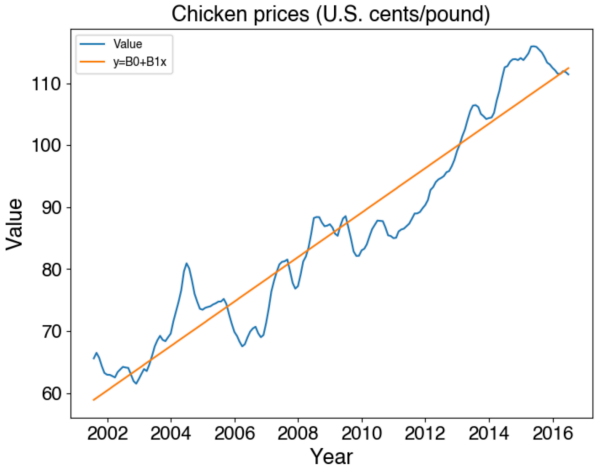
```python
ci_linreg = linreg.conf_int(alpha=0.05)
print(ci_linreg)
print(f'95% confidence interval for price fluctuation: [{ci_linreg[1,0]:.2f},
{ci_linreg[1,1]:.2f}]')
```

```
[[57.19590866 59.97056185]
 [ 0.28604822  0.31263657]]
95% confidence interval for price fluctuation: [0.29, 0.31]
```

结果可视化:

Python

```python
plt.figure(figsize=(8,6))
plt.plot(chicken_data.index, chicken_data['Value'], label='Value')
plt.plot(chicken_data.index, linreg.fittedvalues, label='y=B0+B1x')
plt.xlabel('Year', fontsize=18)
plt.ylabel('Value', fontsize=18)
plt.title('Chicken prices (U.S. cents/pound)', fontsize=18)
plt.xticks(fontsize=16)
plt.yticks(fontsize=16);
plt.legend()
```



模型诊断:

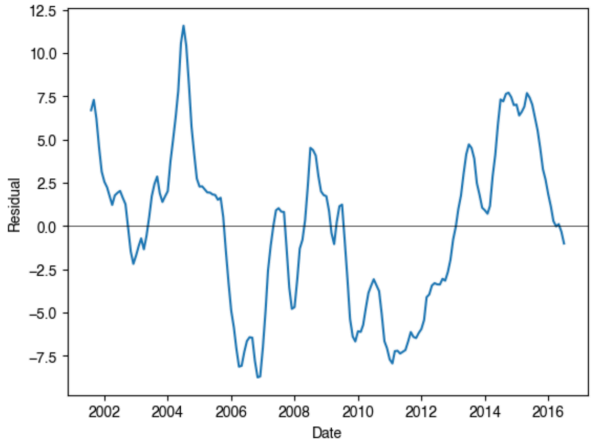
作出 residuals 及其 autocorrelation function 进行模型诊断:

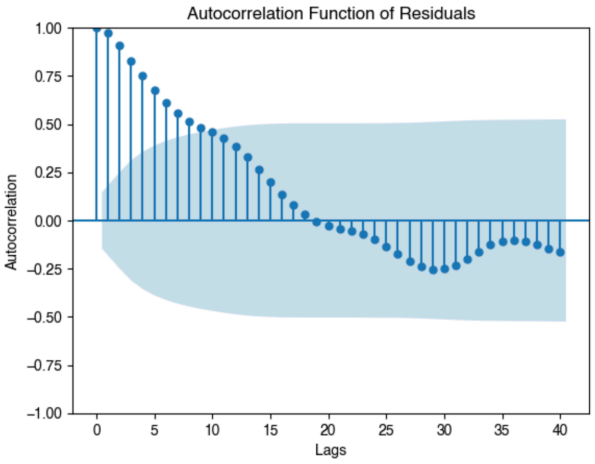
Python

```python
residuals = linreg2.resid

plt.figure()
plt.plot(chicken_data.index,residuals)
plt.axhline(0, color='k', linewidth=0.5) # Horizontal line at 0
plt.xlabel('Date')
plt.ylabel('Residual')

plt.figure(figsize=(10, 5))
plot_acf(residuals, lags=40, title='Autocorrelation Function of Residuals')
plt.xlabel('Lags')
plt.ylabel('Autocorrelation')
plt.show()
```


12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
Residual
2002
2004
2006
2008
2010
2012
2014
2016
Date


Autocorrelation Function of Residuals
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Autocorrelation
0
5
10
15
20
25
30
35
40
Lags